

Contents

1	DDTA - Working Metamodel - Functional Requirement	1
1.1	1. Status and purpose	1
1.2	2. Evidence boundary	1
1.2.1	2.1 Admitted construction evidence	1
1.2.2	2.2 Holdout protection	1
1.3	3. Fundamental question	1
1.4	4. Working semantic definition	2
1.4.1	4.1 Why “independently assessable”	2
1.4.2	4.2 Why “one coherent” function	2
1.5	5. Candidate semantic shape	2
1.5.1	5.1 Candidate-field status	2
1.6	6. The central vertical relation is still open	3
1.6.1	H-FR-PARENT-A - Mandatory Decision parent	3
1.6.2	H-FR-PARENT-B - MR ownership plus Decision applicability	3
1.6.3	6.1 Direct MR identifier may be derived	4
1.7	7. Candidate FR invariants	4
1.7.1	FR-CAND-01 - Functional obligation	4
1.7.2	FR-CAND-02 - Independent assessability	4
1.7.3	FR-CAND-03 - Coherent requirement unit	4
1.7.4	FR-CAND-04 - Decision non-repetition	4
1.7.5	FR-CAND-05 - Implementation independence	4
1.7.6	FR-CAND-06 - Realization is a relation, not FR content	4
1.7.7	FR-CAND-07 - Verification is distinct from realization	4
1.7.8	FR-CAND-08 - Specialized constraints do not replace the FR	4
1.7.9	FR-CAND-09 - Effective obligation is derived context	5
1.7.10	FR-CAND-10 - Corpus non-redundancy candidate	5
1.8	8. Realization and verification boundary	5
1.8.1	8.1 RealizationArtifact is intentionally generic	5
1.9	9. Specialized Requirement boundary	5
1.9.1	9.1 Security-specific future origin	6
1.10	10. Independent facial-access probe	6
1.10.1	10.1 Existing upper-layer context	6
1.10.2	10.2 FR proposition A - acquisition suitable for verification	6
1.10.3	10.3 FR proposition B - governed verification result	6
1.10.4	10.4 Implementation is separate	7
1.11	11. ThreatForge construction probes	7
1.11.1	11.1 Scaffold Requirement	7
1.11.2	11.2 Canonical profiles Requirement	7
1.12	12. Authoring diagnostics for the construction phase	7
1.13	13. What Revision 1 deliberately does not decide	7
1.14	14. Construction sequence from this draft	8
1.15	15. Revision 1 working summary	8

1 DDTA - Working Metamodel - Functional Requirement

NON-CANONICAL WORKING DRAFT - REVISION 1

1.1 1. Status and purpose

This document starts the Functional Requirement (FR) semantic study after the Macro Requirement and Decision models reached a provisional upper-layer baseline.

The purpose is not to reproduce ThreatForge’s current Requirement schema. It is to identify the smallest conceptual model that explains what a Functional Requirement means, how it relates vertically to Macro Requirements and

Decisions, and how it connects downstream to realization and verification without embedding implementation into requirement semantics.

For this phase, **Requirement means Functional Requirement unless explicitly qualified**. Governance, Security and Privacy are deferred specialized-requirement studies.

The current upper-layer baseline remains reopenable. A concrete FR counterexample may require a correction to MR or Decision semantics or relations, followed by regression over prior evidence.

1.2 2. Evidence boundary

1.2.1 2.1 Admitted construction evidence

ThreatForge product-semantic evidence is read at commit:

```
cae0f7b6b37f430ac4e857aabf6ef9f87c89dbb1
```

Construction evidence is limited to ThreatForge MR-0001 and MR-0002 plus the already admitted independent facial-access MR/Decision example.

Selected historical sources used to seed questions, not to define truth, are the active Functional Requirement model/profile, the MR-0001/MR-0002 Requirement registries, and two representative bodies. Their filenames are:

- `functional-requirement.model.yml`
- `functional-requirement-body.profile.yml`
- `MR-0001.requirements.registry.yml`
- `MR-0002.requirements.registry.yml`
- `MR-0001ADR-0007REQ-0001_body.md`
- `MR-0002ADR-0003REQ-0001_body.md`

All are under the canonical `docs/reference/project-model/` tree at the pinned ThreatForge commit above.

ThreatForge currently describes the Functional Requirement model as an “independently testable governed obligation” and gives every FR registry record both a Macro Requirement ID and Decision ID. Its Markdown profile requires **Intent**, **Functional obligation**, **Scope** and **Acceptance** sections. These are historical design choices to test, not automatically adopted L1 semantics.

1.2.2 2.2 Holdout protection

Do not inspect Requirement content belonging to ThreatForge MR-0003, MR-0004 or MR-0005 while constructing this model. Their unconsumed Decision/Requirement families remain future vertical holdout evidence.

1.3 3. Fundamental question

The working FR question is:

What must be observably or objectively true of the governed project/system so that a macro concern and its applicable choices become operational, without prescribing the concrete implementation that makes it true?

This question deliberately separates:

MacroRequirement

major concern / intended result

Decision

significant choice that narrows the concern

FunctionalRequirement

functional truth/capability/behavior/result that must hold

Realization

concrete artifacts and mechanisms that make it hold

Verification

evidence that supports the claim that it holds

1.4 4. Working semantic definition

FR-DEF-01 - Functional Requirement, working definition

A Functional Requirement is a governed, independently assessable obligation stating one coherent capability, behavior, service, state transition or result that the governed project/system must provide or make true. It may operationalize project intent and applicable Decisions, but it does not prescribe the concrete realization mechanism unless that mechanism is itself intrinsic to the required externally/operationally meaningful function.

This definition is a construction hypothesis, not closure.

1.4.1 4.1 Why “independently assessable”

A Requirement should permit a meaningful determination of satisfaction that is stronger than “some implementation artifact exists”. Assessment may ultimately use tests, checks, inspection or other governed evidence, but the exact verification-evidence model is downstream.

1.4.2 4.2 Why “one coherent” function

A Requirement may need multiple normative statements to define one capability. Multiple sentences do not automatically imply multiple Requirements.

Use the split test:

If two obligation groups can be realized, accepted or changed independently and neither is necessary to define the same indivisible functional capability, review them as separate Functional Requirements.

This mirrors the Decision coherence test without assuming one bullet equals one semantic entity.

1.5 5. Candidate semantic shape

Revision 1 intentionally does **not** freeze the historical ThreatForge body sections as the FR semantic core.

FunctionalRequirement

```
| - title?           [likely common governed-document concern]
| - functionalObligation [strong L1 candidate]
| - scope?           [candidate; requires evidence]
| - acceptance?      [candidate; may belong to verification contract]
```

Intent, present in the historical ThreatForge representation, is not yet admitted to the minimum semantic core. Its value must be tested against information already recoverable from the parent context plus the functional obligation.

1.5.1 5.1 Candidate-field status

Candidate	Revision 1 status	Question
title	common identity/display candidate	Is title intrinsic FR semantics or common GovernedDocument semantics?
functionalObligation	strong required candidate	Can any valid FR exist without stating what must be true/provided?
scope	open	Does scope carry independent semantic information, or can it be expressed by the obligation/context without repetition?

Candidate	Revision 1 status	Question
acceptance	open	Is acceptance intrinsic Requirement semantics or a separate verification contract/evidence relation?
intent	open / suspected non-core	Does FR-local intent add non-redundant meaning after MR + Decision + obligation are known?

No representation ordering, Markdown heading, normative keyword or YAML member is L1 merely because ThreatForge currently enforces it.

1.6 6. The central vertical relation is still open

Historical ThreatForge encodes:

MR -> Decision -> FunctionalRequirement

with one `decision_id` and one `macro_requirement_id` in each FR registry record. The current logical model treats both as relations.

Revision 1 does **not** close that topology yet.

Two competing hypotheses must be tested.

1.6.1 H-FR-PARENT-A - Mandatory Decision parent

MacroRequirement

1

|

*

Decision

1

|

*

FunctionalRequirement

Every FR is the operational downstream expression of exactly one significant governed choice.

Potential advantage: strong vertical trace and simple ownership.

Potential defect: it may force artificial Decisions for functions that follow directly from the Macro Requirement/domain obligation without an unresolved choice.

1.6.2 H-FR-PARENT-B - MR ownership plus Decision applicability

MacroRequirement 1 ---- * FunctionalRequirement

Decision * ---- * FunctionalRequirement

constrains / is operationalized by

Every FR has one macro-concern owner, while zero or more Decisions may constrain or be operationalized by it.

Potential advantage: does not require dummy Decisions.

Potential defect: may weaken the simple descending hierarchy and requires an explicit model for applicable Decision context.

The construction and independent examples must decide between these hypotheses or expose a smaller third relation model.

1.6.3 6.1 Direct MR identifier may be derived

Even if H-FR-PARENT-A survives, a stored `macro_requirement_id` may be a derived/indexed mirror of the Decision parent rather than a second independent semantic parent relation.

Identity serialization must not be used as proof of topology.

1.7 7. Candidate FR invariants

These are working tests. Only FR-CAND-01 through FR-CAND-05 are strong enough to guide initial authoring; none is closed before regression.

1.7.1 FR-CAND-01 - Functional obligation

An FR states a required capability, behavior, service, transition or result. Pure rationale, architecture preference, implementation task or evidence record is not an FR.

1.7.2 FR-CAND-02 - Independent assessability

There must be a meaningful way to decide whether the required functional truth holds. “Code exists” or “file created” is not sufficient unless file creation itself is the externally/operationally required behavior.

1.7.3 FR-CAND-03 - Coherent requirement unit

The FR owns one coherent functional capability. Split independently realizable/acceptable capability families.

1.7.4 FR-CAND-04 - Decision non-repetition

An FR does not merely restate the parent/applicable Decision. The Decision explains which choice was made; the FR states what must consequently function or be true.

1.7.5 FR-CAND-05 - Implementation independence

The FR does not prescribe frameworks, source paths, internal class/function layout, code annotations, transport technology or other realization detail unless that mechanism is itself part of the required observable contract.

1.7.6 FR-CAND-06 - Realization is a relation, not FR content

Implementation artifacts may realize an FR, but their existence, paths and implementation state belong to realization trace/governance rather than the intrinsic FR meaning.

1.7.7 FR-CAND-07 - Verification is distinct from realization

Evidence supporting satisfaction is not the same thing as an artifact that realizes the Requirement.

1.7.8 FR-CAND-08 - Specialized constraints do not replace the FR

Security, Governance and Privacy Requirements may enrich/constrain an FR. The underlying functional capability remains owned by the FR.

1.7.9 FR-CAND-09 - Effective obligation is derived context

For realization/review, the effective obligation of an FR includes the FR plus all applicable specialized Requirements. This is a derived governed view, not a reason to copy specialized constraints into the FR body.

1.7.10 FR-CAND-10 - Corpus non-redundancy candidate

Two FRs that express the same functional truth under the same effective governed context may indicate duplication or incorrect decomposition. A corpus-level invariant analogous to Decision H-CLOSED-02 is plausible but remains open until multiple FR families are analyzed.

1.8 8. Realization and verification boundary

A Functional Requirement should lead toward implementation without becoming implementation.

Conceptually:

```
FunctionalRequirement
|
| realizedBy
v
RealizationArtifact *
```

and independently:

```
Requirement
|
| verifiedBy
v
VerificationEvidence *
```

Both cardinalities are deliberately open at L1 because an authored Requirement may exist before implementation/evidence. Lifecycle may later impose stronger conditions for states such as implemented/accepted.

One implementation artifact may realize several Requirements, and one Requirement may require several artifacts. Therefore no one-file-per-FR assumption is admitted.

1.8.1 8.1 RealizationArtifact is intentionally generic

The portable metamodel should not assume that realization means source code only. A functional capability may be realized by code, configuration, infrastructure, a composed service, or another project artifact.

ThreatForge may later support specific artifact classes without making those classes universal FR semantics.

1.9 9. Specialized Requirement boundary

Revision 1 reserves the following future structure without closing its common semantics:

```
FunctionalRequirement
|
+-- GovernanceRequirement *
+-- SecurityRequirement *
`-- PrivacyRequirement *
```

The current working interpretation is:

A Specialized Requirement adds an obligation/constraint that the realization and/or verification of its Functional Requirement must satisfy, without replacing the functional capability owned by the FR.

Whether **specializes**, **constrains**, **appliesTo** or another relation name best represents this meaning remains open.

1.9.1 9.1 Security-specific future origin

Security is not created merely by the FR hierarchy. The later Security model must also preserve analytical provenance:

```
AnalysisRecord
|
v
CommonFinding
|
| accepted
v
```

```

SecurityRequirement
  |
  v
FunctionalRequirement

```

The Security Requirement states a product/project obligation. STRIDE or STRIDE-AI remains recoverable through the Analysis Record rather than becoming a field of the FR.

1.10 10. Independent facial-access probe

The facial-access example deliberately creates a vertical probe that may challenge mandatory Decision parentage.

1.10.1 10.1 Existing upper-layer context

Macro concern (facial-access example): facial recognition should determine whether the acquired face corresponds to the expected authorized person with sufficient quality for the access process.

Existing admitted Decision proposition from the Decision study: the recognition result consumed by the access process has stable governed semantics independent from raw/model-specific ML score semantics.

1.10.2 10.2 FR proposition A - acquisition suitable for verification

Working obligation

The system must acquire facial information at the access point that is sufficient to attempt person verification and must make an insufficient acquisition distinguishable from a valid verification result.

This looks locally like a Functional Requirement: it states a functional capability and can be assessed without naming a camera model, library or source file.

But its parentage is intentionally unresolved. It may follow directly from the Macro Requirement rather than from the specific Decision about stable result semantics.

Therefore this FR is a direct test of H-FR-PARENT-A versus H-FR-PARENT-B.

1.10.3 10.3 FR proposition B - governed verification result

Working obligation

For every completed verification attempt, the system must expose a result using the governed verification semantics consumed by the access process rather than exposing raw model-specific score semantics as the functional result.

This proposition appears to operationalize the existing stable-semantics Decision much more directly.

The pair is useful because both are legitimate functions under the same macro concern, while only one is obviously downstream of the known Decision.

1.10.4 10.4 Implementation is separate

Possible realization artifacts such as a camera adapter, capture service, face-verification service or score interpreter are not part of either FR definition.

Likewise, tests for insufficient capture or stable result mapping are verification evidence, not the FR itself.

1.11 11. ThreatForge construction probes

1.11.1 11.1 Scaffold Requirement

Historical MR-0002ADR-0003REQ-0001 states a coherent user/tool capability - create a governed implementation scaffold - but its current Functional obligation also includes details such as specific source declarations, a deterministic artifact identifier, implementation-trace mutation and invocation through a VS Code task.

The FR study must classify these statements rather than assume the whole historical body is one semantic Requirement:

- required functional behavior;
- consequence of an architectural Decision;
- realization/trace mechanism;
- specialized Governance constraint;
- separate FR that can change independently.

This is a strong construction case for FR-CAND-03, FR-CAND-04 and FR-CAND-05.

1.11.2 11.2 Canonical profiles Requirement

Historical MR-0001ADR-0007REQ-0001 requires canonical logical models and representation profiles for active governed document models. It is a useful contrast because its obligation is closer to a product/tool capability than to a concrete implementation path, yet several bullets may still represent separately changeable obligations.

It therefore probes the semantic granularity of one FR versus a family of FRs.

1.12 12. Authoring diagnostics for the construction phase

Use these questions when rewriting or authoring candidate FRs:

1. **Functional truth:** What must actually work or be true?
2. **Choice test:** Is this sentence still a choice/trade-off? If yes, it may belong to Decision.
3. **Implementation test:** Could a different implementation satisfy the same sentence? If not, is the named mechanism genuinely part of the required contract or merely realization detail?
4. **Split test:** Could one obligation change/be accepted without the other? If yes, consider separate FRs.
5. **Parent test:** Which MR/Decision context naturally owns this function? Would forcing one Decision parent create a dummy Decision?
6. **Verification test:** What observation could establish satisfaction without treating artifact existence as proof?
7. **Specialization test:** Is this the function itself, or an additional Security/Governance/Privacy constraint on that function?
8. **Copy test:** Is the FR repeating a Decision, controlled vocabulary or specialized constraint that should be referenced/effectively inherited instead?

1.13 13. What Revision 1 deliberately does not decide

- whether every FR has exactly one Decision parent;
- whether FR has an independent semantic MR relation or only a derived/mirrored one;
- whether **Intent** is an FR field;
- whether **Scope** is semantically required;
- whether acceptance conditions are intrinsic FR semantics or a separate verification contract;
- the exact relation names/cardinalities for realization and verification;
- FR lifecycle/status;
- FR-to-FR dependencies;
- the common superclass semantics of Governance/Security/Privacy;
- Privacy-specific origin semantics;
- Security-specific Finding cardinality beyond the already observed future derivation requirement;
- any YAML/Markdown representation profile;
- any ThreatForge implementation mechanism.

1.14 14. Construction sequence from this draft

Revision 1 FR hypotheses

|

v

inventory every Functional Requirement in ThreatForge MR-0001/MR-0002

|


```

      v
classify each statement:
FR semantics / Decision / realization / verification / specialization / representation
|
      v
test parent hypothesis and requirement granularity
|
      v
author independent facial-access FR corpus
|
      v
vertical MR -> Decision -> FR regression
|
+--> no counterexample: strengthen/close candidates
|
`--> counterexample: correct smallest owning layer and regress prior evidence

```

Only after the Functional Requirement boundary is stable should the study define the shared `SpecializedRequirement` semantics and then test Security/Governance/Privacy individually.

1.15 15. Revision 1 working summary

The smallest current hypothesis is intentionally modest:

```

FunctionalRequirement
= governed functional obligation
+ independently assessable satisfaction
+ no intrinsic implementation prescription

```

The major unresolved structural question is whether the vertical topology is truly:

```
MR -> Decision -> FR
```

or whether the evidence requires a weaker relation in which MR owns the FR and Decisions constrain/are operationalized by it.

That question should be answered by evidence before the existing ThreatForge ID grammar or registry relations are promoted into the DDTA conceptual metamodel.